

Multi-Agent System — Google ADK version

This notebook rebuilds the plain-Python multi-agent system on top of **Google's Agent Development Kit (ADK)**. The same team collaborates to research a topic and write a short, reviewed report.

Agent	Responsibility	ADK building block
Planner	Breaks the user's goal into concrete subtasks	LlmAgent + output_schema=Plan
Researcher	Answers each subtask using tools	LlmAgent + tools=[...] (ADK runs the tool loop)
Writer	Turns research notes into a report; revises using feedback	LlmAgent + output_key="draft"
Critic	Approves the draft or requests changes	LlmAgent + output_schema=Review
Orchestrator	Routes messages, manages state, enforces the revision loop	custom BaseAgent

How the original pieces map onto ADK:

Original (plain Python)	Google ADK
AnthropicLLM / MockLLM with complete()	A model string ("gemini-3.5-flash") or any BaseLlm subclass — the mock is a BaseLlm
MessageBus + Message	ADK events (session.events) — every model reply, tool call and tool result is an event
SharedMemory blackboard	Session state (session.state), written via output_key and EventActions(state_delta=...)
TOOLS dict + JSON {"tool": ...} protocol	Plain Python functions passed as tools= — ADK reads the docstrings and does native function calling
extract_json()	output_schema=<Pydantic model> — ADK validates the JSON for you
Orchestrator.run()	Orchestrator._run_async_impl() driven by an ADK Runner

1. Setup

Runs offline with a deterministic mock by default. Set GOOGLE_API_KEY (or configure Vertex AI) to use a real Gemini model.

```
# !pip install google-adk -q

import os, json, re, time, asyncio, warnings
from typing import AsyncGenerator, Optional
from collections import Counter

from pydantic import BaseModel, Field
from google.genai import types
from google.adk.agents import BaseAgent, LlmAgent
from google.adk.agents.invocation_context import InvocationContext
from google.adk.events import Event, EventActions
from google.adk.models import BaseLlm, LlmRequest, LlmResponse
from google.adk.runners import InMemoryRunner

warnings.filterwarnings("ignore", message=r".*\[EXPERIMENTAL\].*")

USE MOCK = not (os.environ.get("GOOGLE_API_KEY") or os.environ.get("GOOGLE_GENAI_USE_VERTEXAI"))
MODEL = "gemini-3.5-flash"
MAX_REVISIONS = 3
APP_NAME = "research_team"

print("Mode:", "MOCK (offline)" if USE MOCK else f"LIVE ({MODEL})")

Mode: MOCK (offline)
```

2. Models

In ADK an agent's model is either a model-name string (Gemini) or a BaseLlm instance. The offline mock is simply a BaseLlm subclass that returns the same canned outputs as the original MockLLM — but in ADK's native format, so the Researcher emits real function calls rather than JSON text.

To keep using Claude instead of Gemini, change make_model to return LiteLlm(model="anthropic/claude-sonnet-5-5") (from google.adk.models.lite_llm, needs pip install litellm).

```
def _text_of(content: types.Content) -> str:
    return "".join(p.text or "" for p in (content.parts or []))

class MockLlm(BaseLlm):
```

```

"""Deterministic stand-in that imitates each agent's expected output.

ADK talks to every model through BaseLlm.generate_content_async, so a mock is
just another BaseLlm subclass. One instance per role keeps the logic simple.
"""
model: str = "mock"
role: str = ""
calls: int = 0

async def generate_content_async(
    self, llm_request: LlmRequest, stream: bool = False
) -> AsyncGenerator[LlmResponse, None]:
    self.calls += 1
    last = llm_request.contents[-1] if llm_request.contents else None
    last_text = _text_of(last) if last else ""
    yield LlmResponse(
        content=types.Content(role="model", parts=self._reply(llm_request, last, last_text)),
        usage_metadata=types.GenerateContentResponseUsageMetadata(
            prompt_token_count=0, candidates_token_count=0, total_token_count=0),
    )

def _reply(self, req: LlmRequest, last, last_text: str) -> list[types.Part]:
    if self.role == "Planner":
        return [types.Part(text=json.dumps({"subtasks": [
            "Define what solar energy is and how photovoltaic cells work",
            "Find current cost and efficiency figures for solar panels",
            "Estimate yearly output of a 5 kW home system",
        ]}))]

    if self.role == "Researcher":
        # Second step: the last content is the tool's result -> give the final finding.
        responses = [p.function_response for p in (last.parts or []) if p.function_response]
        if responses:
            obs = responses[0].response.get("result", responses[0].response)
            return [types.Part(text=f"Finding: {obs}")]
        # First step: choose a tool based on the subtask.
        if "Estimate" in last_text:
            call = types.FunctionCall(name="calculator", args={"expr": "5 * 4.5 * 365"})
        elif "cost" in last_text:
            call = types.FunctionCall(name="kb_search", args={"query": "solar cost efficiency"})
        else:
            call = types.FunctionCall(name="kb_search", args={"query": "photovoltaic"})
        return [types.Part(function_call=call)]

    if self.role == "Writer":
        draft = ("# Solar Energy at a Glance\n\n"
            "Solar panels use photovoltaic cells to turn sunlight into electricity. "
            "Costs have fallen sharply while efficiency keeps rising. "
            "A 5 kW home system produces roughly 8,200 kWh per year.")
        if "FEEDBACK" in last_text:
            draft += ("\n\n## Key numbers\n- Typical panel efficiency: ~20-23%\n"
                "- Module prices fell ~90% between 2010 and 2020\n"
                "- Assumes ~4.5 peak sun hours per day\n\n"
                "***Bottom line:** for most sunny homes, solar now pays for itself.")
        return [types.Part(text=draft)]

    if self.role == "Critic":
        if self.calls == 1:
            review = {"approved": False, "score": 6,
                "feedback": "Add concrete numbers (efficiency, price trend), state the sun-hour assumption, and end w"}
        else:
            review = {"approved": True, "score": 9, "feedback": "Clear, sourced and well structured."}
        return [types.Part(text=json.dumps(review))]

    return [types.Part(text="OK")]

def make_model(role: str, use_mock: bool = USE MOCK):
    return MockLlm(role=role) if use_mock else MODEL

```

3. Tools

Tools are ordinary Python functions. ADK builds the function declaration from the **type hints and docstring**, so the docstrings now matter: they're what the model reads to decide which tool to call. The hand-rolled JSON protocol and the ReAct loop in

`ResearcherAgent.run` are no longer needed — ADK runs that loop itself.

```
KNOWLEDGE_BASE = {
    "photovoltaic": "Photovoltaic (PV) cells are semiconductor devices, usually silicon, that convert sunlight directly into electricity via the photovoltaic effect.",
    "solar cost efficiency": "Commercial panel efficiency is typically 20-23%. Module prices fell roughly 90% between 2010 and 2020.",
    "wind": "Modern onshore wind turbines are typically rated 2-5 MW.",
}

def kb_search(query: str) -> str:
    """Searches the internal knowledge base for facts about energy topics.

    Args:
        query: Keywords to search for, e.g. "photovoltaic" or "solar cost efficiency".

    Returns:
        Matching passages joined together, or "No results.".
    """
    q = query.lower()
    hits = [v for k, v in KNOWLEDGE_BASE.items() if any(w in k for w in q.split())]
    return " ".join(hits) if hits else "No results."

def calculator(expr: str) -> str:
    """Evaluates a plain arithmetic expression.

    Args:
        expr: An arithmetic expression using digits, + - * / and parentheses, e.g. "5 * 4.5 * 365".

    Returns:
        The expression and its result, or an error message.
    """
    if not re.fullmatch(r"[\d\s+\-*/()\.]+", expr):
        return "Error: only arithmetic is allowed."
    try:
        return f"{expr} = {eval(expr, {'__builtins__': {}}, {}):,.2f}"
    except Exception as e:
        return f"Error: {e}"

def word_count(text: str) -> str:
    """Counts the words in a piece of text.

    Args:
        text: The text to count.

    Returns:
        The number of words, as a string.
    """
    return str(len(text.split()))

TOOLS = [kb_search, calculator, word_count]

print(calculator("5 * 4.5 * 365"))
print(kb_search("photovoltaic"))
```

5 * 4.5 * 365 = 8,212.50

Photovoltaic (PV) cells are semiconductor devices, usually silicon, that convert sunlight directly into electricity via the photovoltaic effect.

4. Structured outputs

Instead of asking for JSON in the prompt and parsing it with `extract_json`, the Planner and Critic declare a Pydantic `output_schema`. ADK validates the reply and stores the resulting dict in session state under `output_key`.

```
class Plan(BaseModel):
    subtasks: list[str] = Field(description="2-4 small, independent research subtasks")

class Review(BaseModel):
    approved: bool
    score: int = Field(ge=1, le=10)
    feedback: str
```

5. Agents

Each agent is an `LlmAgent` with an instruction (the old system prompt), optional tools, and an `output_key` that writes its final answer into **session state** — this replaces `SharedMemory.write`.

All four use `include_contents="none"`: instead of seeing the whole session history, each agent only sees the message the Orchestrator just sent it (plus its own tool calls in that turn). That matches the original design, where every agent got a freshly built prompt.

```
def build_agents(use_mock: bool = USE MOCK) -> dict[str, LlmAgent]:
    planner = LlmAgent(
        name="Planner",
        model=make_model("Planner", use_mock),
        description="Breaks the user's goal into research subtasks.",
        instruction="You are a planning agent. Break the user's goal into 2-4 small, independent research subtasks.",
        output_schema=Plan,
        output_key="plan",
        include_contents="none",
    )
    researcher = LlmAgent(
        name="Researcher",
        model=make_model("Researcher", use_mock),
        description="Answers one research subtask using tools.",
        instruction=(
            "You are a research agent. Use your tools to answer the subtask you are given, "
            "then reply with one concise finding in plain text. "
            "kb_search searches a knowledge base; calculator does arithmetic; word_count counts words."
        ),
        tools=TOOLS,
        output_key="last_finding",
        include_contents="none",
    )
    writer = LlmAgent(
        name="Writer",
        model=make_model("Writer", use_mock),
        description="Turns research notes into a Markdown report.",
        instruction=(
            "You are a writer agent. Write a short, well-structured Markdown report (under 250 words) "
            "using ONLY the research notes provided. If FEEDBACK is given, revise the previous draft to address it."
        ),
        output_key="draft",
        include_contents="none",
    )
    critic = LlmAgent(
        name="Critic",
        model=make_model("Critic", use_mock),
        description="Reviews drafts and approves or requests changes.",
        instruction=(
            "You are a strict reviewer. Judge the draft for accuracy against the notes, clarity, "
            "use of concrete numbers, and a clear conclusion."
        ),
        output_schema=Review,
        output_key="review",
        include_contents="none",
    )
    return dict(planner=planner, researcher=researcher, writer=writer, critic=critic)
```

6. Orchestrator

The control flow (plan → research each subtask → write ⇌ review) is dynamic: the number of subtasks is only known after planning, and the loop exits on the Critic's verdict. A custom `BaseAgent` expresses that in plain Python, exactly like the original `Orchestrator.run`:

```
User goal → Planner → [Researcher × N subtasks] → Writer ⇌ Critic (≤ MAX_REVISIONS) → Final report
```

Rather than calling agents with a prompt string, the Orchestrator **yields a message event** addressed to the next agent (recipient stored in `custom_metadata`) and then runs that agent. State updates are emitted as `EventActions(state_delta=...)`, which is how ADK expects state to change so it's persisted and auditable.

```
class Orchestrator(BaseAgent):
    """Custom ADK agent that controls the workflow in plain Python.
```

```

User goal -> Planner -> [Researcher x N] -> Writer <-> Critic (<= max_revisions)
"""
planner: LlmAgent
researcher: LlmAgent
writer: LlmAgent
critic: LlmAgent
max_revisions: int = MAX_REVISIONS

def __init__(self, planner, researcher, writer, critic, max_revisions: int = MAX_REVISIONS):
    super().__init__(
        name="Orchestrator",
        planner=planner, researcher=researcher, writer=writer, critic=critic,
        max_revisions=max_revisions,
        sub_agents=[planner, researcher, writer, critic],
    )

# -- helpers -----
def _say(self, ctx: InvocationContext, to: str, text: str, kind: str = "task",
        state_delta: Optional[dict] = None) -> Event:
    """Orchestrator -> agent message. Because the agents use include_contents='none',
    this message is the start of their 'current turn', i.e. exactly what they see."""
    return Event(
        author=self.name,
        invocation_id=ctx.invocation_id,
        branch=ctx.branch,
        content=types.Content(role="model", parts=[types.Part(text=text)]),
        actions=EventActions(state_delta=state_delta or {}),
        custom_metadata={"to": to, "kind": kind},
    )

def _update(self, ctx: InvocationContext, **state_delta) -> Event:
    """A content-less event whose only job is to write to session state."""
    return Event(author=self.name, invocation_id=ctx.invocation_id, branch=ctx.branch,
        actions=EventActions(state_delta=state_delta))

async def _run_async_impl(self, ctx: InvocationContext) -> AsyncGenerator[Event, None]:
    state = ctx.session.state
    goal = _text_of(ctx.user_content) if ctx.user_content else state.get("goal", "")
    yield self._update(ctx, goal=goal, started_at=time.time(),
        research=[], drafts=[], reviews=[])

    # 1) Plan
    yield self._say(ctx, "Planner", f"Goal: {goal}")
    async for ev in self.planner.run_async(ctx):
        yield ev
    subtasks = state["plan"]["subtasks"]

    # 2) Research each subtask (sequential; see 'Where to go next' for parallel)
    for st in subtasks:
        yield self._say(ctx, "Researcher", f"Subtask: {st}")
        async for ev in self.researcher.run_async(ctx):
            yield ev
        research = state["research"] + [{"subtask": st, "finding": state.get("last_finding", "")}]
        yield self._update(ctx, research=research)

    # 3) Write <-> Review loop
    notes = "\n".join(f"- {r['subtask']}: {r['finding']}" for r in state["research"])
    feedback, review, round_ = None, {}, 0
    for round_ in range(1, self.max_revisions + 1):
        prompt = f"Goal: {goal}\n\nResearch notes:\n{notes}"
        if feedback:
            prompt += f"\n\nPrevious draft:\n{state['draft']}\n\nFEEDBACK: {feedback}"
        yield self._say(ctx, "Writer", prompt, state_delta={"round": round_})
        async for ev in self.writer.run_async(ctx):
            yield ev

        yield self._say(ctx, "Critic", f"Goal: {goal}\n\nResearch notes:\n{notes}\n\nDraft:\n{state['draft']}")
        async for ev in self.critic.run_async(ctx):
            yield ev
        review = state["review"]
        yield self._update(ctx, drafts=state["drafts"] + [state["draft"]],
            reviews=state["reviews"] + [review])
        if review.get("approved"):
            break
        feedback = review.get("feedback")

```

```

yield self._say(ctx, "User", "Final report delivered.", kind="system", state_delta={
    "report": state["draft"],
    "approved": review.get("approved", False),
    "score": review.get("score"),
    "rounds": round_,
    "seconds": round(time.time() - state["started_at"], 2),
})

```

7. Event log (replaces the message bus)

ADK already records every step as an `Event`. This helper turns an event into the same `sender → recipient (kind): text` rows that `MessageBus` printed.

```

def describe(event: Event) -> list[tuple[str, str, str, str]]:
    """Turn an ADK event into (sender, recipient, kind, text) rows, like the old MessageBus."""
    rows = []
    meta = event.custom_metadata or {}
    for call in event.get_function_calls():
        rows.append((event.author, f"tool:{call.name}", "tool", json.dumps(call.args)))
    for resp in event.get_function_responses():
        rows.append((f"tool:{resp.name}", event.author, "tool", str(resp.response.get("result", resp.response))))
    if event.content and any(p.text for p in event.content.parts or []):
        text = _text_of(event.content)
        if meta.get("to"):
            rows.append((event.author, meta["to"], meta.get("kind", "task"), text))
        else:
            kind = "feedback" if event.author == "Critic" else "result"
            rows.append((event.author, "Orchestrator", kind, text))
    return rows

def print_event(event: Event):
    for sender, to, kind, text in describe(event):
        preview = text.replace("\n", " ")
        preview = preview[:110] + ("..." if len(preview) > 110 else "")
        print(f"{sender:>16} → {to:<16} ({kind}): {preview}")

```

8. Run the system

The `Runner` drives the root agent and an `InMemorySessionService` stores events and state. Jupyter supports top-level `await`.

```

async def run_team(goal: str, use_mock: bool = USE MOCK, verbose: bool = True,
                  max_revisions: int = MAX_REVISIONS):
    agents = build_agents(use_mock)
    root = Orchestrator(**agents, max_revisions=max_revisions)
    runner = InMemoryRunner(agent=root, app_name=APP_NAME)
    session = await runner.session_service.create_session(app_name=APP_NAME, user_id="user")

    message = types.Content(role="user", parts=[types.Part(text=goal)])
    async for event in runner.run_async(user_id="user", session_id=session.id, new_message=message):
        if verbose:
            print_event(event)

    session = await runner.session_service.get_session(app_name=APP_NAME, user_id="user", session_id=session.id)
    s = session.state
    result = {
        "goal": goal, "report": s["report"], "approved": s["approved"], "score": s["score"],
        "rounds": s["rounds"], "events": len(session.events), "seconds": s["seconds"],
    }
    return result, session

```

```
result, session = await run_team("Write a short briefing on home solar energy: how it works, costs, and expected output.")
```

Orchestrator → Planner	(task): Goal: Write a short briefing on home solar energy: how it works, costs, and expected
Planner → Orchestrator	(result): {"subtasks": ["Define what solar energy is and how photovoltaic cells work", "Find
Orchestrator → Researcher	(task): Subtask: Define what solar energy is and how photovoltaic cells work
Researcher → tool:kb_search	(tool): {"query": "photovoltaic"}
tool:kb_search → Researcher	(tool): Photovoltaic (PV) cells are semiconductor devices, usually silicon, that convert sun
Researcher → Orchestrator	(result): Finding: Photovoltaic (PV) cells are semiconductor devices, usually silicon, that
Orchestrator → Researcher	(task): Subtask: Find current cost and efficiency figures for solar panels
Researcher → tool:kb_search	(tool): {"query": "solar cost efficiency"}

```

tool:kb_search → Researcher
Researcher → Orchestrator
Orchestrator → Researcher
Researcher → tool:calculator
tool:calculator → Researcher
Researcher → Orchestrator
Orchestrator → Writer
Writer → Orchestrator
Orchestrator → Critic
Critic → Orchestrator
Orchestrator → Writer
Writer → Orchestrator
Orchestrator → Critic
Critic → Orchestrator
Orchestrator → User
(tool): Commercial panel efficiency is typically 20-23%. Module prices fell roughly 90% betw
(result): Finding: Commercial panel efficiency is typically 20-23%. Module prices fell rough
(task): Subtask: Estimate yearly output of a 5 kW home system
(tool): {"expr": "5 * 4.5 * 365"}
(tool): 5 * 4.5 * 365 = 8,212.50
(result): Finding: 5 * 4.5 * 365 = 8,212.50
(task): Goal: Write a short briefing on home solar energy: how it works, costs, and expected
(result): # Solar Energy at a Glance Solar panels use photovoltaic cells to turn sunlight i
(task): Goal: Write a short briefing on home solar energy: how it works, costs, and expected
(feedback): {"approved": false, "score": 6, "feedback": "Add concrete numbers (efficiency, p
(task): Goal: Write a short briefing on home solar energy: how it works, costs, and expected
(result): # Solar Energy at a Glance Solar panels use photovoltaic cells to turn sunlight i
(task): Goal: Write a short briefing on home solar energy: how it works, costs, and expected
(feedback): {"approved": true, "score": 9, "feedback": "Clear, sourced and well structured."
(system): Final report delivered.

```

```
from IPython.display import Markdown, display
```

```

print(f"Approved: {result['approved']} | Score: {result['score']}/10 | "
      f"Rounds: {result['rounds']} | Events: {result['events']} | Time: {result['seconds']}s\n")
display(Markdown(result["report"]))

```

```
Approved: True | Score: 9/10 | Rounds: 2 | Events: 30 | Time: 2.85s
```

Solar Energy at a Glance

Solar panels use photovoltaic cells to turn sunlight into electricity. Costs have fallen sharply while efficiency keeps rising. A 5 kW home system produces roughly 8,200 kWh per year.

Key numbers

- Typical panel efficiency: ~20–23%
- Module prices fell ~90% between 2010 and 2020
- Assumes ~4.5 peak sun hours per day

Bottom line: for most sunny homes, solar now pays for itself.

9. Inspect what happened

```

# Session state (the old shared memory)
for key in ["plan", "research", "drafts", "reviews"]:
    value = session.state.get(key)
    n = len(value) if isinstance(value, list) else 1
    print(f"{key}: {n} entr{'y' if n == 1 else 'ies'}")

print("\nPlan:", json.dumps(session.state["plan"]["subtasks"], indent=2))
print("\nAll reviews:")
for r in session.state["reviews"]:
    print("  ", r)

```

```

plan: 1 entry
research: 3 entries
drafts: 2 entries
reviews: 2 entries

```

```

Plan: [
  "Define what solar energy is and how photovoltaic cells work",
  "Find current cost and efficiency figures for solar panels",
  "Estimate yearly output of a 5 kW home system"
]

```

```

All reviews:
{'approved': False, 'score': 6, 'feedback': 'Add concrete numbers (efficiency, price trend), state the sun-hour assumption, a
{'approved': True, 'score': 9, 'feedback': 'Clear, sourced and well structured.'}

```

```

# Who talked to whom? An interaction matrix built from the ADK events.
edges = Counter((src.split(":")[0], dst.split(":")[0])
                for ev in session.events for src, dst, _, _ in describe(ev))
print(f"{'FROM':<14}{'TO':<14}COUNT")
for (src, dst), n in sorted(edges.items(), key=lambda x: -x[1]):
    print(f"{src:<14}{dst:<14}{n}")

```

FROM	TO	COUNT
Orchestrator	Researcher	3
Researcher	tool	3

tool	Researcher	3
Researcher	Orchestrator	3
Orchestrator	Writer	2
Writer	Orchestrator	2
Orchestrator	Critic	2
Critic	Orchestrator	2
user	Orchestrator	1
Orchestrator	Planner	1
Planner	Orchestrator	1
Orchestrator	User	1

Optional: draw the communication graph (needs matplotlib + networkx)

try:

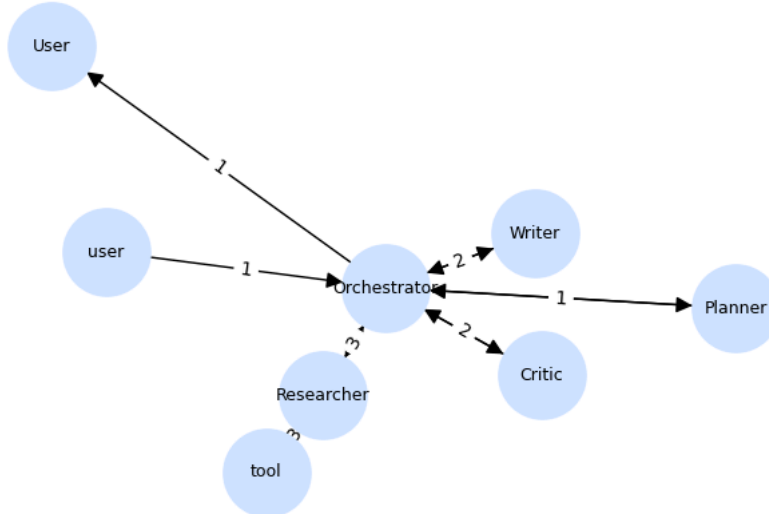
```
import networkx as nx
import matplotlib.pyplot as plt
```

```
G = nx.DiGraph()
for (src, dst), n in edges.items():
    G.add_edge(src, dst, weight=n)
pos = nx.spring_layout(G, seed=7)
plt.figure(figsize=(8, 5))
nx.draw_networkx(G, pos, node_color="#cfe3ff", node_size=2200, font_size=9, arrowsize=18)
nx.draw_networkx_edge_labels(G, pos, edge_labels={(u, v): d["weight"] for u, v, d in G.edges(data=True)})
plt.title("Agent communication graph"); plt.axis("off"); plt.show()
```

except ImportError:

```
print("Install networkx and matplotlib to see the graph: pip install networkx matplotlib")
```

Agent communication graph



10. Simple tests

The mock is deterministic, so we can assert on behaviour — handy for CI.

```
async def test_pipeline():
    out, sess = await run_team("Write a short briefing on home solar energy.", use_mock=True, verbose=False)
    assert out["approved"] is True
    assert out["rounds"] == 2, "Critic should reject once, then approve"
    assert len(sess.state["research"]) == 3
    assert "Key numbers" in out["report"]
    assert any(dst.startswith("tool:") for ev in sess.events for _, dst, _, _ in describe(ev)), \
        "Researcher should use tools"

def test_tools():
    assert "8,212.50" in calculator("5 * 4.5 * 365")
    assert calculator("__import__('os')").startswith("Error")
    assert "silicon" in kb_search("photovoltaic")

await test_pipeline(); test_tools()
print("All tests passed ✅")
```

All tests passed ✅

11. Where to go next

- **Parallel research:** build one Researcher per subtask (unique names, `output_key=f"finding_{i}"`) and run them with `ParallelAgent`, or `asyncio` inside the Orchestrator.
- **Declarative workflows:** newer ADK releases offer a graph-based `Workflow` API (`google.adk.workflow`) as the successor to `SequentialAgent` / `LoopAgent`; the custom-agent approach here works across ADK versions.
- **Real tools:** swap `kb_search` for ADK's built-in `google_search`, a vector store, or an MCP server via `McpToolset`.
- **Dynamic routing:** give an `LlmAgent` coordinator the others as `sub_agents` and let it decide who acts next via ADK's agent transfer.
- **Persistence:** replace `InMemoryRunner`'s session service with `DatabaseSessionService` so runs can be resumed and audited.
- **Dev UI:** put the agents in a package exposing `root_agent` and run `adk web` to inspect every event and state change in a browser.